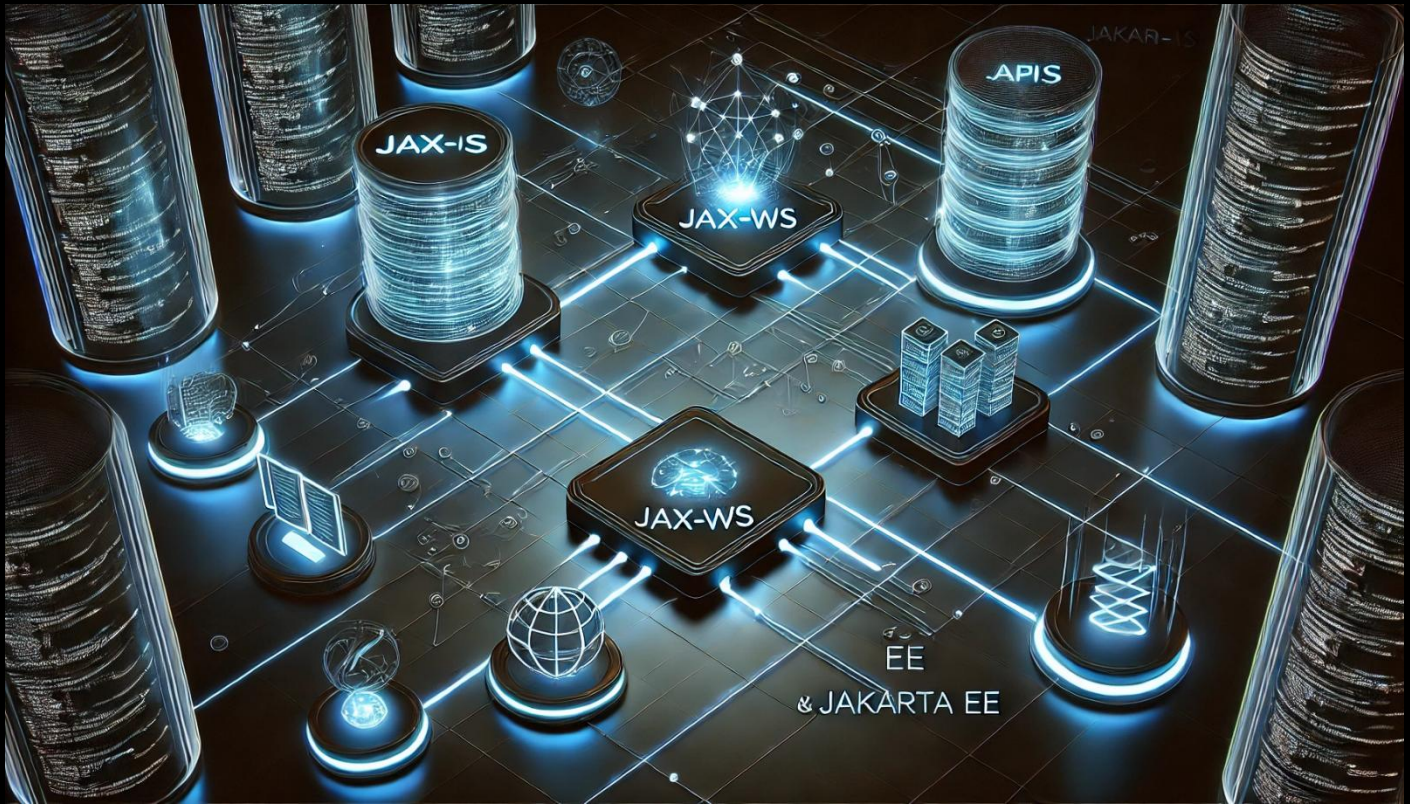


Cliente del Web Service con JAX-WS y Jakarta EE



Guía Paso a Paso: Creación del Cliente para Consumir un Web Service con JAX-WS

Introducción

En esta lección aprenderemos a consumir un servicio web (Web Service) desde un cliente en Java utilizando la especificación **JAX-WS**.

El cliente llamará a un método de suma expuesto por un servicio web que ya fue implementado en la lección anterior.

1. Crear el Proyecto Cliente

1. En NetBeans, haz clic en **File > New Project**.
2. Selecciona **Java with Maven > Java Application**.
3. El proyecto se llamará: `ClienteSumaWS`

4. Ubícalo dentro de la carpeta C:\Cursos\Java\JakartaEE,
5. El Group ID: `sga`
6. Omite el nombre del paquete principal.
7. Haz clic en **Finish**.

 **Ruta del proyecto creado:** C:\Cursos\Java\JakartaEE\ClienteSumaWS

2. Agregar Dependencias

2.1. Dependencia de Jakarta EE API

```
<dependency>
  <groupId>jakarta.platform</groupId>
  <artifactId>jakarta.jakartaee-api</artifactId>
  <version>11.0.0-M1</version>
  <scope>provided</scope>
</dependency>
```

2.2. Dependencia de JAX-WS para Web Services

```
<dependency>
  <groupId>jakarta.jws</groupId>
  <artifactId>jakarta.jws-api</artifactId>
  <version>3.0.0</version>
</dependency>
```

 Agregamos algunas otras dependencias como `slf4j` y otras dependencias para poder generar el cliente WSDL

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>2.0.13</version>
  <type>jar</type>
</dependency>
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-simple</artifactId>
  <version>2.0.13</version>
  <scope>runtime</scope>
</dependency>
<!-- Dependencia de transporte HTTP para CXF -->
```

```

<dependency>
  <groupId>org.apache.cxf</groupId>
  <artifactId>cxf-rt-transports-http</artifactId>
  <version>4.1.1</version>
</dependency>
<dependency>
  <groupId>jakarta.xml.ws</groupId>
  <artifactId>jakarta.xml.ws-api</artifactId>
  <version>4.0.2</version>
</dependency>
<dependency>
  <groupId>com.sun.xml.ws</groupId>
  <artifactId>jaxws-rt</artifactId>
  <version>4.0.3</version>
</dependency>

```

3. Configurar Plugin para Generar Cliente desde WSDL

Vamos a generar automáticamente las clases cliente a partir del archivo WSDL del servicio web.

Agrega el siguiente plugin dentro de <build><plugins> en tu pom.xml:

```

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.cxf</groupId>
      <artifactId>cxf-codegen-plugin</artifactId>
      <version>4.1.1</version>
      <executions>
        <execution>
          <id>generate-sources</id>
          <goals>
            <goal>wsdl2java</goal>
          </goals>
          <configuration>
            <wsdlOptions>
              <wsdlOption>
                <wsdl>
http://localhost:8080/ServicioSumarImplService/ServicioSumarImpl?wsdl
                </wsdl>
              </wsdlOption>
            </wsdlOptions>
            <extraargs>
              <extraarg>-p</extraarg>
              <extraarg>clientews.servicio</extraarg>
            </extraargs>
          </configuration>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>

```

```

        <outputDir>${basedir}/src/main/java</outputDir>
    </wsdlOption>
</wsdlOptions>
</configuration>
</execution>
</executions>
</plugin>
</plugins>
</build>

```

 Las clases generadas se depositarán automáticamente en:
/ClienteSumaWS/src/main/java/clientews/servicio/

Archivo pom.xml completo:

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>sga</groupId>
    <artifactId>ClienteSumaWS</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>
    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>21</maven.compiler.release>
        <exec.mainClass>ClienteSumaWS</exec.mainClass>
    </properties>
    <dependencies>
        <dependency>
            <groupId>jakarta.platform</groupId>
            <artifactId>jakarta.jakartaee-api</artifactId>
            <version>11.0.0-M1</version>
            <scope>provided</scope>
        </dependency>
        <dependency>
            <groupId>jakarta.jws</groupId>
            <artifactId>jakarta.jws-api</artifactId>
            <version>3.0.0</version>
        </dependency>
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-api</artifactId>
            <version>2.0.13</version>

```

```

        <type>jar</type>
    </dependency>
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-simple</artifactId>
        <version>2.0.13</version>
        <scope>runtime</scope>
    </dependency>
    <!-- Dependencia de transporte HTTP para CXF -->
    <dependency>
        <groupId>org.apache.cxf</groupId>
        <artifactId>cxf-rt-transports-http</artifactId>
        <version>4.1.1</version>
    </dependency>
    <dependency>
        <groupId>jakarta.xml.ws</groupId>
        <artifactId>jakarta.xml.ws-api</artifactId>
        <version>4.0.2</version>
    </dependency>
    <dependency>
        <groupId>com.sun.xml.ws</groupId>
        <artifactId>jaxws-rt</artifactId>
        <version>4.0.3</version>
    </dependency>
</dependencies>
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.cxf</groupId>
            <artifactId>cxf-codegen-plugin</artifactId>
            <version>4.1.1</version>
            <executions>
                <execution>
                    <id>generate-sources</id>
                    <goals>
                        <goal>wsdl2java</goal>
                    </goals>
                    <configuration>
                        <wsdlOptions>
                            <wsdlOption>
                                <wsdl>
http://localhost:8080/ServicioSumarImplService/ServicioSumarImpl?wsdl</wsdl>
                                <extraargs>
                                    <extraarg>-p</extraarg>
                                    <extraarg>clientews.servicio</extraarg>
                                </extraargs>
                            </wsdlOption>
                        </wsdlOptions>
                    </configuration>
                </execution>
            </executions>
        </plugin>
    </plugins>

```

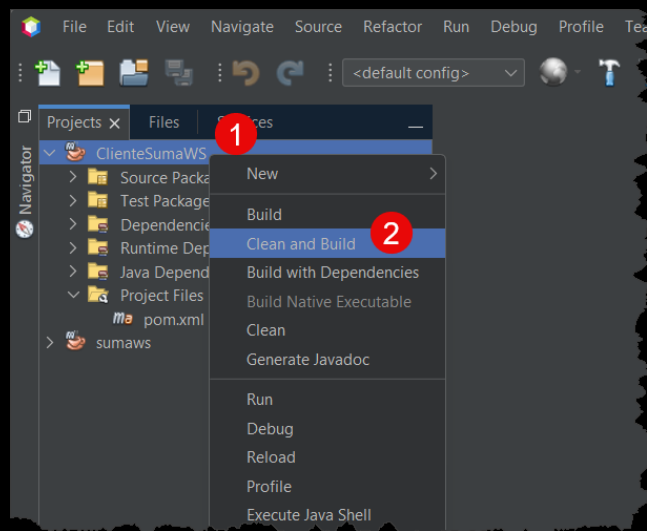
```

        </extraargs>
        <outputDir>${basedir}/src/main/java</outputDir>
    </wsdlOption>
</wsdlOptions>
</configuration>
</execution>
</executions>
</plugin>
</plugins>
</build>
</project>

```

🚀 4. Generar Clases Cliente desde WSDL

1. Haz clic derecho sobre el proyecto `ClienteSumaWS`
2. Selecciona **Clean & Build**



3. Esto descargará las librerías necesarias, y se generarán de manera automática las clases del cliente del servicio web

✅ Esto generará clases como:

- `ServicioSumarImplService.java`
- `ServicioSumarWS.java`
- `Sumar.java`
- `SumarResponse.java`
- Entre otras

5. Crear Clase de Prueba para Consumir el Servicio

 Archivo:

/ClienteSumaWS/src/main/java/testclientews/TestServicioSumarWS.java

```
package testclientews;

import clientews.servicio.ServicioSumarImplService;
import clientews.servicio.ServicioSumarWS;

public class TestServicioSumarWS {

    public static void main(String[] args) {
        ServicioSumarWS servicioSumar = new ServicioSumarImplService().getServicioSumarImplPort();
        System.out.println("Ejecutando Servicio sumar WS");
        int x = 6;
        int y = 3;
        int resultado = servicioSumar.sumar(x, y);
        System.out.println("Resultado: " + resultado);
        System.out.println("Fin servicio sumar WS");
    }
}
```

6. Ejecutar Cliente

1. Haz clic derecho sobre la clase TestServicioSumarWS.java
2. Selecciona **Run File**

 Resultado esperado en consola:

```
Ejecutando servicio sumar WS
Resultado: 9
Fin de servicio sumar WS
```

Esto confirma que el cliente consumió exitosamente el servicio web remoto usando JAX-WS.

Conclusión

En esta guía aprendiste a:

- Crear un proyecto Maven cliente
- Configurar dependencias y plugins para JAX-WS
- Generar clases automáticamente desde WSDL

- Consumir un servicio web real desde un cliente Java

✨ **Este procedimiento es aplicable a cualquier WSDL**, basta con cambiar la URL y el paquete destino.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)